

Bases de Dados

TRANSAÇÕES

©José Barateiro 2024
Adaptado de Silberschatz, Korth e Sudarschan

Conceito de transação

- Uma **transação** é uma *unidade* de execução que acede e, possivelmente, atualiza vários itens de dados.
- Por exemplo, uma transação para transferir €50 da conta A para a conta B:
  1. **read** (*A*)
  2. *A* := *A* – 50
  3. **write** (*A*)
  4. **read** (*B*)
  5. *B* := *B* + 50
  6. **write** (*B*)
- Duas "dificuldades" a serem tratadas:
  - Falhas de vários tipos, como falhas de hardware e falhas de sistema
  - Execução simultânea de múltiplas transações

Exemplo: transferência bancária

- Transação para transferir €50 da conta A para a conta B:
  1. **read** (*A*)
  2. *A* := *A* – 50
  3. **write** (*A*)
  4. **read** (*B*)
  5. *B* := *B* + 50
  6. **write** (*B)*

- **Requisito de atomicidade**
  - Se a transação falhar após a etapa 3 e antes da etapa 6, o dinheiro será "perdido", levando a um estado inconsistente da BD
    - A falha pode ser devida a software ou a hardware
  - O sistema deve garantir que as atualizações de uma transação parcialmente executada não sejam refletidas na BD

- **Requisito de durabilidade** — uma vez que o utilizador tenha sido notificado de que a transação foi concluída (ou seja, a transferência dos €50 ocorreu), as atualizações na BD realizadas pela transação devem persistir mesmo que hajam quaisquer falhas de sistema.

Exemplo de transferência bancária (Cont.)

- **Requisito de consistência** no exemplo acima:
  - A soma de A e B permanece inalterada pela execução da transação

- Em geral, os requisitos de consistência incluem
  - Restrições de integridade especificadas explicitamente, como chaves primárias e chaves estrangeiras
  - Restrições de integridade implícitas
    - por exemplo, a soma dos saldos de todas as contas deve ser consistente após uma transação
  - Durante a execução da transação, a BD pode ficar temporariamente inconsistente.
  - Quando a transação for concluída com sucesso, a BD deve ser consistente
    - Lógica de transação errada pode levar à inconsistência

Exemplo de transferência bancária (Cont.)

Requisito de isolamento — se entre as etapas 3 e 6, outra transação T2 tiver permissão para aceder à BD parcialmente atualizada, ela verá uma BD inconsistente (a soma *A + B* será menor do que deveria ser).

T1	T2
1. **read**(*A*)	
2. *A* := *A* – 50	
3. **write**(*A*)	
	read(A), read(B), print(A+B)
4. **read**(*B*)	
5. *B* := *B* + 50	
6. **write**(*B*)	

- O isolamento pode ser garantido trivialmente executando transações **em série**
  - Ou seja, uma após a outra.
- No entanto, executar múltiplas transações simultaneamente traz benefícios de desempenho muito significativos.

Propriedades ACID

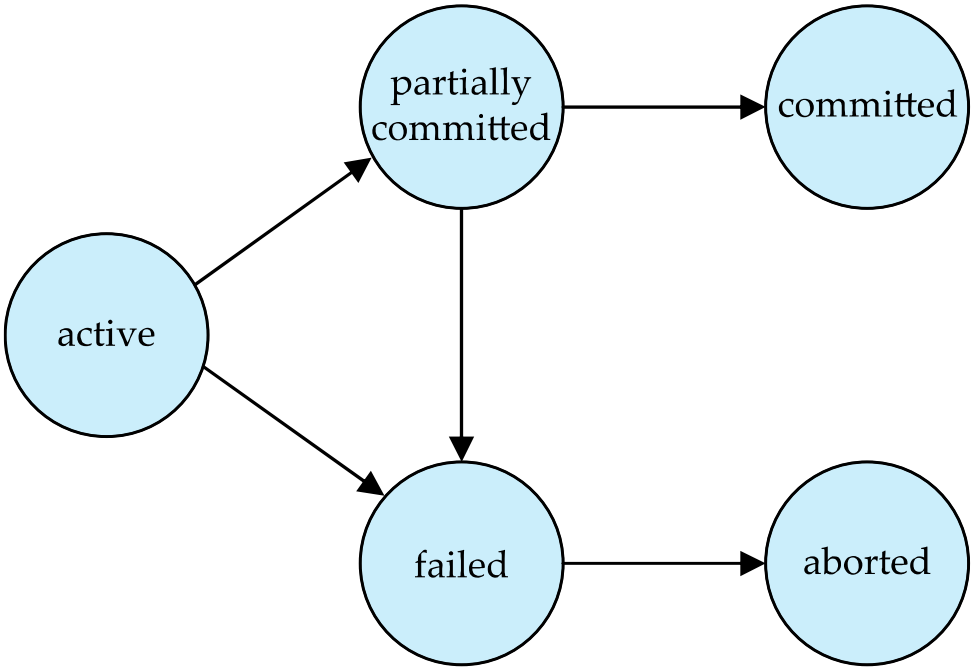
Uma **transação** é uma unidade de execução que acede e, possivelmente, atualiza vários itens de dados. Para preservar a integridade dos dados, o sistema de gestão de BD deve garantir:

- **Atomicidade -** Todas as operações da transação estão refletidas adequadamente na BD, ou nenhuma está.
- **Consistência -** A execução de uma transação isoladamente preserva a consistência da base de dados.
- **Isolamento -** Embora múltiplas transações possam ser executadas simultaneamente, cada transação deve desconhecer as outras transações. Os resultados intermédios da transação devem ser ocultados das outras transações executadas simlutaneamente.
  - Ou seja, para cada par de transações T_i e T_j, parece para T_i que T_j, terminou a execução antes de T_i começar, ou T_j iniciou a execução depois de T_i terminar.
- **Durabilidade -** Depois de uma transação ser concluída com sucesso, as alterações feitas na BD persistem, mesmo que hajam, falhas no sistema.

Estado das transações

- **Active** – estado inicial; a transação permanece neste estado enquanto está em execução
- **Partially committed** – após a instrução final ter sido executada.
- **Failed -** após a descoberta de que a execução normal não pode prosseguir.
- **Aborted** – após a transação ter sido revertida e a BD restaurada para o seu estado anterior ao início da transação.
- **Committed** – após a conclusão bem-sucedida.

Estado das Transações (Cont.)



Execuções Simultâneas

- Várias transações podem ser executadas simultaneamente. As vantagens são:
  - **Maior utilização dos recursos de HW**, levando a um melhor *desempenho das transações*
    - Por exemplo, uma transação usar CPU enquanto a outra está a ler ou a gravar em disco
  - **Tempo médio de resposta reduzido** para transações: as transações curtas não precisam esperar atrás das longas.

Esquemas de controlo de concorrência (concurrency control schemes) são mecanismos para alcançar o isolamento

- Ou seja, controlar a interação entre as transações simultâneas para evitar que destruam a consistência da BD

Schedules

- **Schedule** – uma sequência de instruções que especifica a ordem cronológica em que as instruções de transações simultâneas são executadas
  - Um *schedule* para um conjunto de transações inclui todas as instruções dessas transações
  - Deve preservar a ordem em que as instruções aparecem em cada transação individual.
- Uma transação que conclui a sua execução com sucesso terá uma instrução commit como última instrução
  - Por omissão, a transação assume que executa sempre a instrução de commit como último passo
- Uma transação que não consegue concluir com sucesso a sua execução terá uma instrução de *abort* como última instrução

Schedule 1

- Considere T_1 a transferir €50 de *A* para *B* e T_2 a transferir 10% do saldo de *A* para *B.*
- Um *serial schedule* no qual T_1 é seguido por T_2 :

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
	$B := B + temp$
	write (B)
	commit

Schedule 2

Um *serial schedule* onde T_2 é seguido por T_1

T_1	T_2
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	

Schedule 3

Considere T_1 e T_2 as transações definidas anteriormente. O schedule a seguir não é um serial schedule, mas é *equivalente* ao Schedule 1

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (B)
	$B := B + temp$
	write (B)
	commit

Nos schedules 1, 2 e 3 , a soma A + B é preservada

Schedule 4

O seguinte schedule não preserva o valor de (*A* + *B*) .

T_1	T_2
read (A)	
$A := A - 50$	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	$B := B + temp$
	write (B)
	commit

Serializability

- **Assunção Básica** – Cada transação preserva a consistência da BD.
- Assim, a execução em série de um conjunto de transações preserva a consistência da BD.
- Um schedule (possivelmente simultâneo) é serializável se for equivalente a um serial schedule. Diferentes formas de equivalência de schedules dão origem às noções de:
  1. **Conflict serializability**
  2. **View serializability**

Visão simplificada das transações

- Ignoramos todas as operações além de instruções **de leitura (read)** e **gravação (write)**
- Assumimos que as transações podem realizar cálculos arbitrários nos dados em buffers locais e entre leituras e gravações.
- Os schedules simplificados consistem apenas em instruções **de read** e de **write**.

Instruções conflitantes

- As instruções I_i *e* I_j das transações T_i e T_j respectivamente, **entram em conflito** se e só se existir algum item Q acedido tanto por I_i *como* por I_j, e pelo menos uma dessas instruções escreveu Q.
  1. I_i = **read**(Q), I_j = **read**(Q). Não há conflito
  2. I_i = **read**(Q), I_j = **write**(Q). Conflito
  3. I_i = **write**(Q), I_j = **read**(Q). Conflito
  4. I_i = **write**(Q), I_j = **write**(Q). Conflito

- Intuitivamente, um conflito entre I_i e I_j força uma ordem temporal (lógica) entre eles.

- Se I_i e I_j forem consecutivos num schedule e não entrarem em conflito, os seus resultados permaneceriam os mesmos, mesmo se tivessem sido executados noutra ordem no schedule.

Conflict serializability

- Se um schedule *S* pode ser transformado num schedule *S'* por uma série de trocas de instruções não conflituantes, dizemos que *S* e *S'* são **equivalentes em conflito (conflict equivalent)**.
- Dizemos que um schedule *S* é **conflict serializable** se for equivalente em conflito a um serial schedule.

Conflict serializability (Cont.)

- O Schedule 3 pode ser transformado no Schedule 6, um serial schedule onde T_2 segue T_1, através de trocas de instruções não conflituantes. Assim, o Schedule 3 é conflict serializable.

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 3

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

Schedule 6

Conflict serializability(Cont.)

- Exemplo de um schedule que não é conflict serializable:

T_3	T_4
read (Q)	
	write (Q)
write (Q)	

- Não podemos trocar instruções no schedule acima para obter o serial schedule <T_3, T_4> nem o serial schedule <T_4, T_3>.

View Serializability

- Sejam *S* e *S'* dois schedules com o mesmo conjunto de transações. *S* e *S'* são **view equivalent** se as três condições seguintes se verificam para cada item de dados *Q:*
  1. Se no schedule S a transação T_i lê o valor inicial de *Q* , então no schedule *S'* a transação T_i também lê o valor inicial de *Q.*
  2. Se no schedule S a transação T_i executa **read** (*Q)*, e esse valor foi produzido pela transação T_j (se houver), então no schedule *S'* a transação T_i também deve ler o valor de *Q* que foi produzido pela mesma operação de **write** (Q) da transação T_j.
  3. A transação (se houver) que executa a operação final de **write**(*Q)* no schedule *S* também deve realizar a operação final de **write** (*Q*) no schedule *S'* .

- Como se pode veriticar, **view equivalent** também se baseia apenas nos reads e nos writes.

View Serializability (cont.)

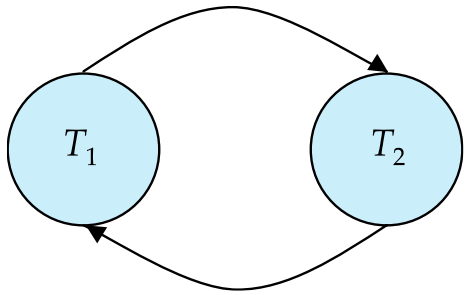
- Um schedule *S* é **view serialiable** se for view equivalent a um serial schedule.
- Todos os schedules **conflict serializable** também são **view serializable**.
- Abaixo está um schedule que é **view serializable**, mas não é **conflict serializable**.

T_{27}	T_{28}	T_{29}
read (Q)		
	write (Q)	
write (Q)		
		write (Q)

- Qual é o serial schedule equivalente ao acima?
- Cada schedule view serializable que não é conflict serializable contém **blind writes**.

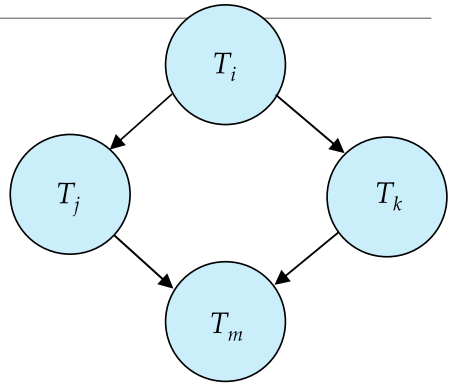
Teste de serialização

- Considere um schedule como um conjunto de transações T_1 , T_2 , ..., T_n
- **Um grafo de precedências** — grafo direto onde os vértices são as transações.
- Desenhamos um arco de T_i para T_j se as duas transações entrarem em conflito, e T_i acedeu primeiro ao item de dados que causa o conflito.
- Podemos rotular o arco com o item de dados que causa o conflito.
- Exemplo de um grafo de precedências

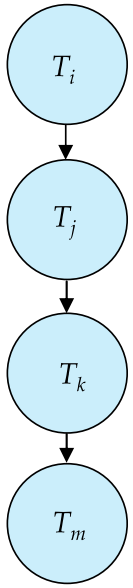


Teste para conflict serializable

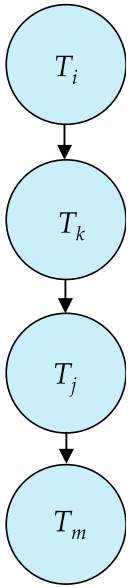
- Um schedule é conflict serializable se e só se o seu grafo de precedências for acíclico.
- Existem algoritmos de detecção de ciclos com complexidade na ordem n^2, onde *n* é o número de vértices no grafo.
  - (Algoritmos otimizados têm complexidade na ordem *n* + *e,* onde *e* é o número de arcos.)
- Se o grafo de precedências for acíclico, a ordem de serialização pode ser obtida através da ordenação topológica do grafo.
  - Esta é uma ordem linear consistente com a ordem parcial do gráfico.
  - Podem haver várias ordenações topológicas



(a)



(b)



(c)

Schedules recuperáveis

Necessidade de abordar o efeito das falhas nas transações em transações executadas simultaneamente.

- **Recoverable schedule** — se uma transação T_j lê um item de dados previamente escrito por uma transação T_i, então a operação de commit de T_i aparece antes da operação de commit de T_j.

- O schedule seguinte não é recuperável

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	

- Se T_8 deveria abortar, T_9 teria lido (e possivelmente mostrado ao utilizador) um estado inconsistente da BD. Portanto, a BD deve garantir que os schedules sejam recuperáveis.

Rollbacks em cascata (Cascading rollbacks)

- **Cascading rollback** – uma única falha numa transação leva a uma série de rollbacks de transações. Considere o seguinte schedule, onde nenhuma das transações foi confirmada (portanto, o schedule é recuperável)

T_{10}	T_{11}	T_{12}
read (A)		
read (B)		
write (A)		
	read (A)	
	write (A)	
		read (A)
abort		

Se T_{10} falhar, T_{11} e T_{12} também deverão ser revertidos.

- Pode levar à anulação de uma quantidade significativa de trabalho

Cascadeless schedules

Cascadeless schedules— rollbaks em cascata não podem ocorrer;

- Para cada par de transações T_i e T_j tais que T_j lê um item de dados previamente escrito por T_i, a operação de commit de T_i aparece antes da operação de leitura de T_j.

- Um cascadeless schedule é também é recuperável
- É desejável restringir os schedules àqueles que sejam cascadeless

Controlo de concorrência

- Uma BD deve fornecer um mecanismo que garanta que todos os schedules possíveis sejam
  - Conflict ou view serializable e
  - são recuperáveis e de preferência cascadeless
- Uma política na qual apenas uma transação pode ser executada de cada vez gera serial schedules, mas fornece um baixo grau de simultaneidade
- Testar a serialização de um schedule *após* sua execução é um pouco tarde demais!

Objetivo – desenvolver protocolos de controlo de concorrência que garantam schedule serializability.

Níveis fracos de consistência

- Algumas aplicações estão dispostas a conviver com níveis fracos de consistência, permitindo schedules que não são serializáveis.
- Por exemplo:
  - uma transação de leitura que deseja obter um saldo total aproximado de todas as contas
  - estatísticas da BD calculadas para otimização de queries podem ser aproximadas
  - Tais transações não precisam de ser serializáveis em relação a outras transações
- Precisão vs desempenho

Níveis de isolamento no SQL-92

Serializable – nível mais alto. As transações são executadas em sequência

Repeatable read- leituras aos mesmos dados não mudam, mesmo que outra transação altere esses dados.

- Evita dirty reads (dados modificados por uma transação não confirmada) e non-repeatable reads (a transação lê o mesmo dados duas vezes e obtém valores diferentes, pois a outra transação já confirmou a modificação).
- Não evita phantom reads (outra transação pode adicionar ou remover linhas que alteram a consulta realizada)
- Bloqueia as linhas que foram lidas

Read commited - apenas registos confirmados podem ser lidos (que não estão em transações pendentes).

- Leituras sucessivas do registo podem retornar valores diferentes (mas confirmados).
- Evita dirty reads
- Não evita non-repeatable reads e phantom reads

Read uncommitted - até mesmo registos não confirmados podem ser lidos.

- Não evita dirty reads, non-repeatable reads e phantom reads

Definição de transação em SQL

- No SQL, uma transação começa implicitamente.
- Uma transação em SQL termina com:
  - **Commit** confirma a transação atual e inicia uma nova.
  - **Rollback** faz com que a transação atual seja abortada.
- Em quase todos os sistemas de gestão de BD, por omissão, cada instrução SQL também é confirmada implicitamente se for executada com êxito
- O nível de isolamento pode ser definido ao nível da BD

  O nível de isolamento pode ser alterado no início da transação
  - Por exemplo, em SQL, **set transaction isolation level serializable**

Implementação de níveis de isolamento

- Locking
  - Bloquear toda a BD vs bloquear itens
  - Quanto tempo para manter o lock?
  - Locks partilhados vs locks exclusivos
- Timestamps
  - Transaction timestamps atribuídos, por exemplo, quando uma transação começa
  - Os itens de dados armazenam dois timestamps
    - Read timestamp
    - Write timestamp
  - Os timestamps são usados para detectar acessos fora de ordem
- Várias versões de cada item de dados
  - Permite que as transações leiam a partir de um "snapshot" da BD